Save as:

`Java_Level_8_Backend_Fundamentals.md`

Java Level 8 — Backend Fundamentals

Overview

Level 8 bridges **Core Java** and **Spring Boot**.

The goal is not to learn every backend technology in depth, but to understand the fundamentals that Spring Boot will later abstract.

By the end of this level, you should understand:

- Gradle
- Java backend project structure
- JDBC
- PostgreSQL connectivity
- Prepared statements
- CRUD
- Transactions
- Connection pooling
- HTTP
- REST APIs
- JSON
- DTOs
- Configuration
- Environment variables
- Logging
- JUnit
- Mockito
- Unit vs Integration testing

---

1. Gradle

Gradle is a **build automation and dependency management tool**.

It handles:

```
Compile code
    ↓
Resolve dependencies
    ↓
Run tests
    ↓
Package application
```

Instead of manually downloading JAR files, Gradle manages dependencies.

1.1 Gradle Project Structure

Typical Java project:

```
todo-backend/
├── build.gradle
├── settings.gradle
├── gradlew
├── gradlew.bat
│
└── src/
    ├── main/
    │   ├── java/
    │   └── resources/
    │
    └── test/
        └── java/
```

Important directories:

```
src/main/java
    → production Java code

src/main/resources
    → configuration/resources

src/test/java
    → tests
```

---

2. `build.gradle`

Example:

```
plugins {
    id 'java'
}

group = 'com.example'
version = '1.0.0'

repositories {
    mavenCentral()
}

dependencies {

    implementation(
        'org.postgresql:postgresql:42.7.7'
    )

    testImplementation(
        'org.junit.jupiter:junit-jupiter:5.12.2'
    )
}

test {
    useJUnitPlatform()
}
```

Important sections:

```
plugins
    → Gradle functionality

repositories
    → dependency sources

dependencies
    → project libraries

test
    → test configuration
```

---

3. `settings.gradle`

Example:

```
rootProject.name = 'todo-backend'
```

For a simple project, this mainly defines the project identity and later can define multi-module structure.

---

4. Gradle Wrapper

Prefer:

```
./gradlew build
```

over:

```
gradle build
```

Difference:

```
gradle build
    ↓
Uses Gradle installed on your machine

./gradlew build
    ↓
Uses the Gradle version configured by the project
```

The wrapper makes builds more reproducible across machines.

Important Gradle Commands

```
./gradlew build
```

Build project.

```
./gradlew test
```

Run tests.

```
./gradlew clean
```

Delete generated build output.

```
./gradlew clean build
```

Clean and rebuild.

```
./gradlew dependencies
```

Display dependency tree.

---

5. Dependency Types

`implementation`

Production dependency:

```
implementation 'org.postgresql:postgresql:42.7.7'
```

`testImplementation`

Dependency required only for tests:

```
testImplementation(
    'org.junit.jupiter:junit-jupiter:5.12.2'
)
```

Mental model:

```
implementation
    → production code

testImplementation
    → tests
```

---

6. JDBC

JDBC = **Java Database Connectivity**.

It provides the standard Java API for communicating with relational databases.

Architecture:

```
Java Application
       ↓
      JDBC
       ↓
PostgreSQL JDBC Driver
       ↓
PostgreSQL
```

Important distinction:

```
JDBC
    → Java database API

PostgreSQL Driver
    → implementation that knows how to communicate
      with PostgreSQL
```

---

7. JDBC Connection

Example:

```
String url =
        "jdbc:postgresql://localhost:5432/todo";

String username = "postgres";
String password = "password";

Connection connection =
        DriverManager.getConnection(
                url,
                username,
                password
        );
```

Connection flow:

```
Java code
    ↓
DriverManager
    ↓
PostgreSQL JDBC Driver
```

```
    ↓
PostgreSQL
```

---

8. PreparedStatement

Parameterized queries should use `PreparedStatement`.

Avoid:

```
String sql =
        "SELECT * FROM users WHERE name = '"
        + name
        + "'";
```

Instead:

```
String sql =
        "SELECT * FROM users WHERE name = ?";
```

Then:

```
PreparedStatement statement =
        connection.prepareStatement(sql);

statement.setString(1, name);
```

Execute:

```
ResultSet resultSet =
        statement.executeQuery();
```

Mental model:

```
SQL template
    ↓
?
    ↓
Bind parameter
    ↓
Execute
```

Benefits:

Benefits:

- Safer SQL
- Prevents SQL injection
- Cleaner parameter handling

---

9. Core JDBC Objects

The most important JDBC objects are:

```
Connection
    ↓
PreparedStatement
    ↓
ResultSet
```

Connection

Represents a database connection.

PreparedStatement

Represents a parameterized SQL statement.

ResultSet

Contains rows returned by a query.

---

10. SELECT

Example:

```
String sql = """
    SELECT id, name
    FROM users
    """;

try (
    Connection connection =
            DatabaseConnection.getConnection();

    PreparedStatement statement =
            connection.prepareStatement(sql);
```

```
    ResultSet resultSet =
            statement.executeQuery()
) {

    while (resultSet.next()) {

        long id =
                resultSet.getLong("id");

        String name =
                resultSet.getString("name");

        System.out.println(
                id + " " + name
        );
    }
}
```

For:

```
SELECT
```

use:

```
executeQuery()
```

---

11. INSERT /UPDATE /DELETE

For:

```
INSERT
UPDATE
DELETE
```

use:

```
executeUpdate()
```

Example:

```
String sql = """
    INSERT INTO todos (title)
```

```
    INSERT INTO todos (title)
    VALUES (?)
    """;

try (
    Connection connection =
            DatabaseConnection.getConnection();

    PreparedStatement statement =
            connection.prepareStatement(sql)
) {

    statement.setString(
            1,
            "Learn Spring Boot"
    );

    int rows =
            statement.executeUpdate();
}
```

The return value is the number of affected rows.

---

12. Try-with-Resources

Database resources should be closed.

Instead of manually:

```
connection.close();
statement.close();
resultSet.close();
```

use:

```
try (
    Connection connection = ...;
    PreparedStatement statement = ...;
    ResultSet resultSet = ...
) {
    // database operations
}
```

Java automatically closes the resources.

Important because leaked database resources can cause:

```
Connection leaks
Resource exhaustion
Database connection limits
```

---

13. JDBC CRUD

CRUD represents the four basic database operations:

```
CREATE → INSERT
READ   → SELECT
UPDATE → UPDATE
DELETE → DELETE
```

Typical Todo API:

```
POST   /todos
GET    /todos
GET    /todos/{id}
PATCH  /todos/{id}
DELETE /todos/{id}
```

---

14. Mapping Database Rows to Java Objects

Example record:

```
public record Todo(
        long id,
        String title,
        boolean completed
) {
}
```

Database row:

```
id | title | completed
```

can be mapped to:

```
Todo todo = new Todo(
```

```
Todo todo = new Todo(
        resultSet.getLong("id"),
        resultSet.getString("title"),
        resultSet.getBoolean("completed")
);
```

Mental model:

```
Database Row
     ↓
ResultSet
     ↓
Java Object
```

Frameworks such as JPA/Hibernate automate much of this mapping.

---

15. Database Transactions

A transaction groups multiple database operations into one atomic unit.

Example: transferring money:

```
Account A
    ↓
- ₹100

Account B
    ↓
+ ₹100
```

Both operations must succeed.

Transaction:

```
BEGIN
  ↓
Debit A
  ↓
Credit B
  ↓
COMMIT
```

If something fails:

```
ROLLBACK
```

```
ROLLBACK
```

JDBC Transaction

```
connection.setAutoCommit(false);

try {

    debitAccount(connection);

    creditAccount(connection);

    connection.commit();

} catch (Exception e) {

    connection.rollback();

    throw e;
}
```

Mental model:

```
Transaction
    ↓
Operation 1
    ↓
Operation 2
    ↓
COMMIT
```

Failure:

```
ROLLBACK
```

Important

Transactions provide **atomicity**:

> Either all required database changes happen, or none of them do.

Spring Boot later makes this much easier with:

```
@Transactional
```

16. Connection Pooling

Creating a new database connection for every request is expensive.

Without pooling:

```
Request
   ↓
Create connection
   ↓
Query
   ↓
Close connection
```

With pooling:

```
            Connection Pool
          ┌──────────────────┐
          │ Connection 1     │
          │ Connection 2     │
          │ Connection 3     │
          │ Connection 4     │
          └──────────────────┘
            ↑            ↓
          borrow       return
            ↑            ↓
            Application
```

Flow:

```
Request
   ↓
Borrow connection
   ↓
Execute query
   ↓
Return connection
```

17. HikariCP

HikariCP is a popular Java JDBC connection pool.

Typical architecture:

```
Application
     ↓
DataSource
     ↓
HikariCP
     ↓
PostgreSQL
```

Spring Boot commonly uses HikariCP as its default connection pool for JDBC/DataSource configurations.

Why Pool Size Matters

Suppose:

```
Pool size = 10
```

and:

```
100 requests
```

arrive simultaneously.

Approximately:

```
10 → database connections
90 → wait for a connection
```

Increasing pool size indefinitely is not a solution because PostgreSQL also has connection limits and database resources are finite.

---

18. HTTP

HTTP is the protocol commonly used for communication between clients and backend servers.

Architecture:

```
Frontend
    ↓
HTTP Request
    ↓
Backend
    ↓
HTTP Response
```

A request contains:

```
Method
URL
Headers
Body
```

Example:

```
POST /todos
Content-Type: application/json

{
    "title": "Learn Spring Boot"
}
```

---

19. HTTP Methods

Important methods:

Method	Purpose
GET	Read
POST	Create
PUT	Replace
PATCH	Partial update
DELETE	Delete

Mental model:

```
GET     → Read
POST    → Create
PUT     → Replace
```

```
PUT     → Replace
PATCH   → Modify partially
DELETE  → Remove
```

---

20. HTTP Status Codes

Important status codes:

Success

```
200 OK
201 Created
204 No Content
```

Client errors

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
```

Server errors

```
500 Internal Server Error
503 Service Unavailable
```

Remember:

```
401 → Authentication problem

403 → Authenticated but not allowed
```

---

21. REST APIs

REST APIs expose resources through HTTP.

Example:

```
GET    /users
GET    /users/10
POST   /users
PATCH  /users/10
DELETE /users/10
```

Prefer resource-oriented endpoints:

```
/users
/users/{id}
```

rather than:

```
/getUsers
/createUser
/deleteUser
```

---

22. JSON

JSON is the most common data format for REST APIs.

Request:

```
{
  "title": "Learn Spring Boot"
}
```

Response:

```
{
  "id": 10,
  "title": "Learn Spring Boot",
  "completed": false
}
```

Typical flow:

```
JSON
 ↓
Java Object
```

and

and:

```
Java Object
 ↓
JSON
```

Spring Boot/Jackson will automate this conversion.

---

23. DTOs

DTO = **Data Transfer Object**.

Example:

```
public record CreateTodoRequest(
        String title
) {
}
```

Response:

```
public record TodoResponse(
        long id,
        String title,
        boolean completed
) {
}
```

DTOs define the API contract.

Why DTOs?

Don't blindly expose database entities.

Database:

```
User
├── id
├── name
├── email
├── password_hash
├── created_at
└── internal_status
```

```
    internal_status
```

API response:

```
UserResponse
├── id
├── name
└── email
```

DTOs help prevent accidental exposure of internal fields.

---

24. Configuration

Avoid hardcoding environment-specific configuration:

```
String password = "myPassword";
```

Instead:

```
Environment
    ↓
Configuration
    ↓
Application
```

Typical environment variables:

```
DB_HOST
DB_PORT
DB_NAME
DB_USERNAME
DB_PASSWORD
```

Development:

```
localhost
```

Production:

```
production database
```

The application code remains the same while configuration changes by environment.

Spring Boot provides excellent support for this.

---

25. Logging

Avoid relying on:

```
System.out.println();
```

Production applications use logging frameworks.

Important log levels:

```
TRACE
DEBUG
INFO
WARN
ERROR
```

Typical usage:

```
DEBUG → detailed developer information
INFO  → normal application events
WARN  → unexpected but recoverable situation
ERROR → failure
```

Example:

```
logger.info(
        "Creating todo title={}",
        title
);
```

Error:

```
logger.error(
        "Failed to create todo",
        exception
);
```

Never Log Secrets

Avoid logging:

```
Passwords
JWT tokens
API keys
Database credentials
Credit card information
Other sensitive secrets
```

---

26. JUnit

JUnit is the standard Java testing framework.

Example:

```
@Test
void shouldAddNumbers() {

    int result = 10 + 20;

    assertEquals(30, result);
}
```

The common testing pattern is:

```
Arrange
   ↓
Act
   ↓
Assert
```

Example:

```
@Test
void shouldCalculateTotal() {

    // Arrange
    Calculator calculator =
            new Calculator();

    // Act
    int result =
```

```
            calculator.add(10, 20);

    // Assert
    assertEquals(30, result);
}
```

---

27. Mockito

Mockito is used to mock dependencies during unit testing.

Suppose:

```
class TodoService {

    private final TodoRepository repository;

    TodoService(TodoRepository repository) {
        this.repository = repository;
    }
}
```

Without mocking:

```
TodoService
    ↓
TodoRepository
    ↓
PostgreSQL
```

With Mockito:

```
TodoService
    ↓
Mock TodoRepository
```

Example:

```
TodoRepository repository =
        mock(TodoRepository.class);
```

Configure behavior:

```
when(repository.findById(1L))
```

```
when(repository.findById(1L))
        .thenReturn(
                Optional.of(todo)
        );
```

Now the service can be tested without a real database.

---

28. Unit vs Integration Tests

Unit Test

Tests a component in isolation.

```
TodoService
    ↓
Mock Repository
```

Characteristics:

```
Fast
Isolated
No real database
```

Integration Test

Tests actual component integration.

```
TodoService
    ↓
Repository
    ↓
PostgreSQL
```

Characteristics:

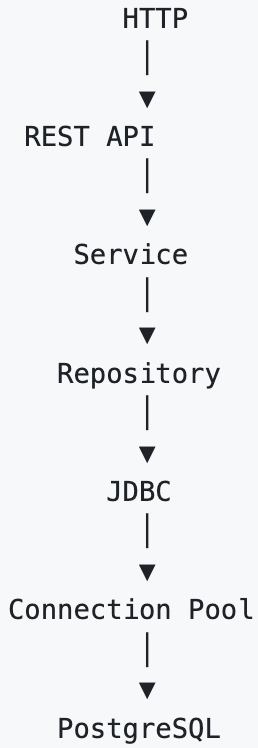
```
Slower
Real dependencies
Tests integration
```

Both are important in production systems.

---

29. Level 8 Architecture

The important backend architecture learned in this level is:



Supporting infrastructure:

```
┌──────────────────────────────┐
│ Gradle                       │
│ Configuration                │
│ Environment Variables        │
│ Logging                      │
│ JUnit                        │
│ Mockito                      │
└──────────────────────────────┘
```

30. Level 8 → Spring Boot Mapping

This is the most important section to remember before Level 9.

What we learned manually:

```
Level 8
```

Spring Boot will simplify:

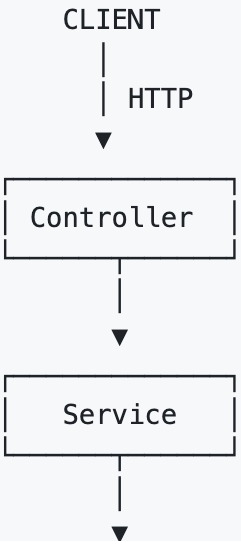
Level 8 Concept	Spring Boot
`DriverManager`	`DataSource`
JDBC connection management	Connection pool/DataSource
Manual SQL mapping	JPA/Hibernate
Manual transactions	`@Transactional`
HTTP server/API handling	Spring MVC
JSON conversion	Jackson
Configuration	`application.yml` / properties
Environment variables	Spring configuration
Manual dependency creation	Dependency Injection
Manual object wiring	Spring IoC
Manual logging setup	Spring Boot logging
JUnit	JUnit + Spring Test
Mockito	Mockito + Spring Test

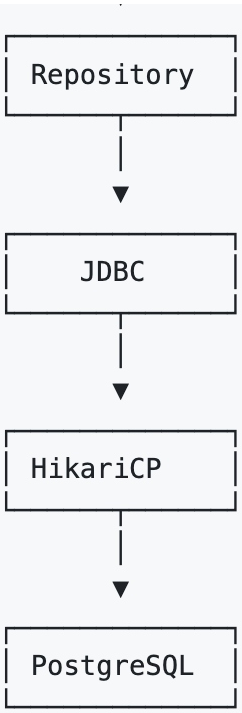
This is why learning the fundamentals first is valuable.

---

31. Core Mental Model

Remember this architecture:





In Spring Boot, much of this becomes:

```
Client
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
JPA/Hibernate
  ↓
DataSource/HikariCP
  ↓
PostgreSQL
```

---

32. Level 8 — Important Patterns

Parameterized SQL

```
String sql =
        "SELECT * FROM todos WHERE id = ?";

statement.setLong(1, id);
```

Try-with-resources

```
try (
    Connection connection = ...;
    PreparedStatement statement = ...;
) {
}
```

Transaction

```
connection.setAutoCommit(false);

try {
    operation1();
    operation2();

    connection.commit();

} catch (Exception e) {

    connection.rollback();
}
```

DTO

```
record CreateTodoRequest(
        String title
) {
}
```

Testing

```
Arrange
 ↓
Act
 ↓
Assert
```

Mocking

```
Service
  ↓
Mock dependency
```

33. Level 8 Final Checklist

```
✅ Gradle
✅ build.gradle
✅ settings.gradle
✅ Gradle Wrapper
✅ implementation
✅ testImplementation

✅ JDBC
✅ PostgreSQL JDBC Driver
✅ Connection
✅ PreparedStatement
✅ ResultSet
✅ executeQuery()
✅ executeUpdate()
✅ Try-with-resources
✅ CRUD
✅ Row → Java Object mapping
✅ Transactions
✅ COMMIT
✅ ROLLBACK
✅ Connection Pooling
✅ HikariCP

✅ HTTP
✅ HTTP methods
✅ HTTP status codes
✅ REST
✅ JSON
✅ DTOs

✅ Environment Variables
✅ Configuration
✅ Logging

✅ JUnit
✅ Mockito
✅ Unit Tests
✅ Integration Tests
```

🎯 What NOT to Memorize

You don't need to memorize:

- Every Gradle task

- Every JDBC class/method
- HTTP specification details
- Every HTTP status code
- Logging framework internals
- Advanced connection-pool configuration

Focus on understanding **how the pieces fit together**.

🚀 Next: Level 9 — Spring Boot

Now we can move quickly.

The key question for Level 9 is:

> **What does Spring actually solve?**

We'll start with:

```
Without Spring

new UserRepository()
        ↓
new UserService(repository)
        ↓
new UserController(service)
```

versus:

```
With Spring

Spring Container
       ↓
creates objects
       ↓
connects dependencies
       ↓
Controller → Service → Repository
```

Then we'll build the actual backend:

```
Spring Boot
     ↓
REST API
     ↓
PostgreSQL
```

```
PostgreSQL
     ↓
JPA/Hibernate
     ↓
Validation
     ↓
Exception Handling
     ↓
Authentication
     ↓
JWT
     ↓
Production-ready API
```